

Workstream v0.1

Architecture Baseline

Governed task execution, immutable evaluation, human review, contribution records and external fulfillment boundaries

ARCHITECTURE STATEMENT Workstream is the authoritative control plane for project policy, tasks, submissions, checker outcomes, review decisions, contributions, compensation awards and audit history. Identity Issuer proves the caller; Flow Node preserves and retrieves evidence; external adapters fulfill awards. None of those external systems may mutate Workstream lifecycle meaning.

Field	Baseline value
Document ID	WS-ARCH-001
Version	Workstream v0.1
Status	Architecture baseline - team review candidate
Architecture style	C4: context, containers, components and deployment
Implementation style	Python/FastAPI modular monolith with durable workers
Date	11 July 2026
Owner	Workstream Engineering

Document control and precedence

AUTHORITY After team approval, WS-ARCH-001 defines the Workstream v0.1 baseline. WS-REV-001 remains the normative review contract and WS-CON-001 remains the normative contribution and compensation-boundary contract. If a summary here is less specific, the lifecycle specification controls within its own scope.

Source	How this baseline uses it
Workstream Architecture Brief	Retains project setup, task policy, submission and checker concepts; removes unrelated systems and superseded economics wording.
WS-POL-001-16 Live API Drill	Defines what has actually been demonstrated through durable checker execution and review_pending.
Identity Issuer v0.1 C4 Draft	Defines the external identity boundary: subject, signed token, JWKS and optional introspection. Product roles stay inside Workstream.
Flow Node Definitive Architecture	Defines production evidence storage, CID/manifests, provenance, pinning and semantic search behind Workstream ports.
WS-REV-001	Locks reviewer grants, routing, FIFO, preferences, leases, immutable reviews, revision chains and reject behavior.
WS-CON-001	Locks contribution creation, compensation-policy freezing, awards, outbox delivery and fulfillment receipts.

Reading guide

Sections 1-5 explain what Workstream is and how its C4 boundaries connect. Sections 6-15 follow the lifecycle and its data, security and reliability rules. Sections 16-19 turn the baseline into deployment, proof and delivery decisions. The companion WS-IMP-001 document is the deterministic coding-agent contract.

Contents

1. Executive summary
2. Scope, responsibilities and non-goals
3. Runtime proof and design maturity
4. Actors, roles and authority
5. C4 Level 1 - system context
6. C4 Level 2 - container architecture
7. C4 Level 3 - FastAPI components
8. C4 Level 3 - asynchronous worker components
9. End-to-end lifecycle
10. Identity and authorization
11. Flow Node evidence boundary

- 12. Canonical domain and data ownership
- 13. Review, revision and contribution lifecycles
- 14. Transactions, concurrency and events
- 15. API and integration architecture
- 16. Security, privacy and audit
- 17. Failure handling, recovery and observability
- 18. C4 deployment view
- 19. Conformance, delivery and baseline decisions

Appendix A. Glossary

1. Executive summary

Workstream v0.1 is a governed contribution system. It converts a project guide into an approved effective policy, binds that policy to task assignments, accepts immutable submission versions, executes deterministic post-submission checks, routes admitted work to authorized reviewers, preserves every review and revision, and records the resulting contribution and project-scoped compensation award.

The baseline is a Python/FastAPI modular monolith. This keeps the lifecycle transactionally coherent while the domain is still being proven. PostgreSQL is the authority for business state. Celery workers execute durable background jobs through Redis. The checker runner is an isolated deterministic execution boundary. A transactional outbox carries evidence and fulfillment work to external adapters without making request handlers depend on network success.

Two external systems are intentional parts of the boundary. Identity Issuer authenticates a portable subject and signs the token; Workstream resolves that subject to a local actor and performs every role and resource check. Flow Node stores immutable artifact bytes, manifests, content identifiers, provenance and semantic indexes; Workstream retains the meaning of those objects through local bindings and lifecycle rules.

MOST IMPORTANT DECISION A token proves identity, an artifact CID proves content, and an adapter receipt proves a downstream response. Only Workstream proves whether an actor was authorized, whether a task changed state, what a review decided, and what contribution or award exists.

1.1 Architectural outcome

- One system of record: PostgreSQL-backed Workstream lifecycle state.
- One implementation language for v0.1: Python, with FastAPI, SQLAlchemy/Pydantic and Celery.
- One canonical human-review chain: immutable SubmissionVersion, Review and ReviewFinding records.
- One clean economics boundary: Workstream records policy, award and receipt; adapters execute money, points or credits.
- One evidence port model: FlowNodeAdapter in production and LocalStorageAdapter only in development and CI.

2. Scope, responsibilities and non-goals

2.1 Workstream responsibilities

Capability	Canonical Workstream responsibility
Project governance	Guide snapshots, sufficiency decisions, effective policies, activation and locked policy references.

Capability	Canonical Workstream responsibility
Task execution	Queueing, claim, assignment, task context, submitter authority and task state.
Evaluation	Immutable submissions, checker runs, review routing, leases, findings, decisions and revision history.
Contribution	Global ContributionRecord identity with project filters and immutable source references.
Compensation boundary	Project-scoped compensation policy versions, award calculation, outbox instructions and fulfillment receipts.
Accountability	Authorization, audit facts, immutable chains, operational projections and reconciliation.

2.2 Deliberate non-goals

- Workstream is not an identity provider. It does not issue human identity or own authentication factors.
- Workstream is not an artifact byte store or global semantic database. It owns lifecycle bindings to evidence.
- Workstream is not a payment provider, points ledger, wallet, balance service or finance-approval engine.
- Workstream is not a general agent runtime or unrelated workspace product.
- Workstream v0.1 does not include an adjudication layer above the reviewer.
- Workstream v0.1 does not reopen a rejected task to another contributor; reject closes the task.

3. Runtime proof and design maturity

The architecture distinguishes observed behavior from design commitments. This prevents a diagram from silently claiming production evidence that does not yet exist.

Area	Status	Evidence or next proof
Project setup and policy derivation	Runtime-proven	WS-POL-001-16 live API drill.
Task locked context and pre-submit rejection	Runtime-proven	Failed pre-submit produced no submission side effect.
Submission finalization and durable checker	Runtime-proven	Successful path reached review_pending with a durable checker run.
Review and revision lifecycle	Locked design	Implement WS-REV-001, then run its conformance and live-drill suites.
Contribution and compensation boundary	Locked design	Implement WS-CON-001, then prove atomic decision-to-record behavior.

Area	Status	Evidence or next proof
Identity Issuer integration	Target architecture	Replace drill HMAC proof with issuer JWKS/token conformance.
Flow Node production adapter	Target architecture	Prove ingest, verification, pinning, retrieval, search and recovery.

PROOF BOUNDARY The existing drill proves the intake spine through review_pending. It does not prove review completion, contribution creation, external award fulfillment, Identity Issuer JWKS, or production Flow Node behavior.

4. Actors, roles and authority

4.1 Major actor domains

Workstream has two human actor domains: Admin and Contributor. A person can hold roles in both domains, but every command is authorized by the exact role and resource scope required for that command.

Actor domain	v0.1 roles	Purpose
Admin	Operator	Run queues, recovery, operational controls and scoped overrides.
Admin	Project Manager	Create projects, approve policies, manage task and project configuration.
Admin	Finance Authority	Publish compensation policy and manage adapter bindings; does not execute provider payment inside Workstream.
Admin	Reputation Authority	Consume contribution facts and manage future reputation policy; scoring is outside this baseline.
Contributor	Submitter	Claim permitted tasks and create submission versions.
Contributor	Reviewer	Claim review queue entries and record decisions within an active project grant.
Contributor	Both	May act as submitter or reviewer where the project grant permits; self-review remains prohibited.

4.2 Authority composition

Authorization is the intersection of four facts: a valid issuer token, an active local Workstream actor, the required product role or ProjectRoleGrant, and the resource guard for the exact project/task/submission. No single fact replaces the others.

- Admin roles are Workstream product roles and may be project-scoped where the command permits it.

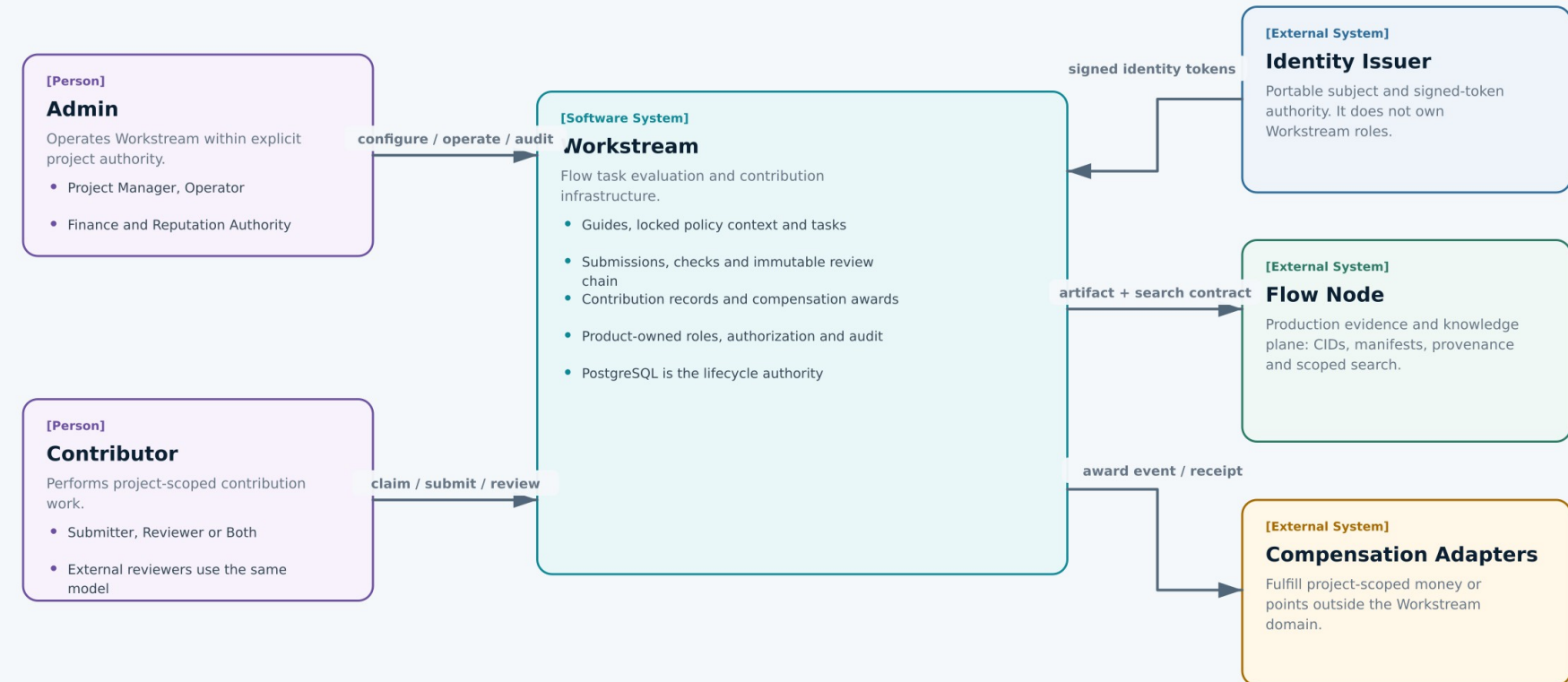
- Submitter/reviewer privilege is represented by ProjectRoleGrant; reputation and skills may inform an admin decision but do not automatically grant v0.1 authority.
- A TaskAssignment ban is task-scoped and is checked in addition to the contributor's project grant.
- External reviewers use the same Workstream contributor and grant model after onboarding; they do not receive a second authority path.

5. C4 Level 1 - system context

The system context treats Workstream as one software system and shows only people and external systems that cross its trust boundary.

C4 Level 1 - System Context

Workstream is the human-facing contribution control plane; external systems provide identity, evidence and fulfillment.



WORKSTREAM v0.1 ARCHITECTURE BASELINE

Figure 1. C4 Level 1 - Workstream system context

5.1 Context connections, one by one

1. Admin to Workstream. Operators and scoped admin roles configure projects, approve policies, manage grants, operate queues and inspect audit evidence through authenticated Workstream APIs and the internal operations UI.

2. Contributor to Workstream. Submitters claim tasks and submit immutable versions. Reviewers claim leased review work and record canonical decisions. The same person may be granted both roles, but the command determines the authority evaluated.

3. Workstream to Identity Issuer. Workstream verifies the signed token, issuer, audience and expiry through issuer JWKS or approved introspection. It then resolves the issuer subject to a local actor. Workstream does not ask the issuer whether the actor is a reviewer or project manager.

4. Workstream to Flow Node. Workstream uses service-authenticated ports for artifact ingest, verification, pinning, retrieval and scoped semantic search. Flow Node returns evidence identifiers and receipts; it cannot accept or reject work.

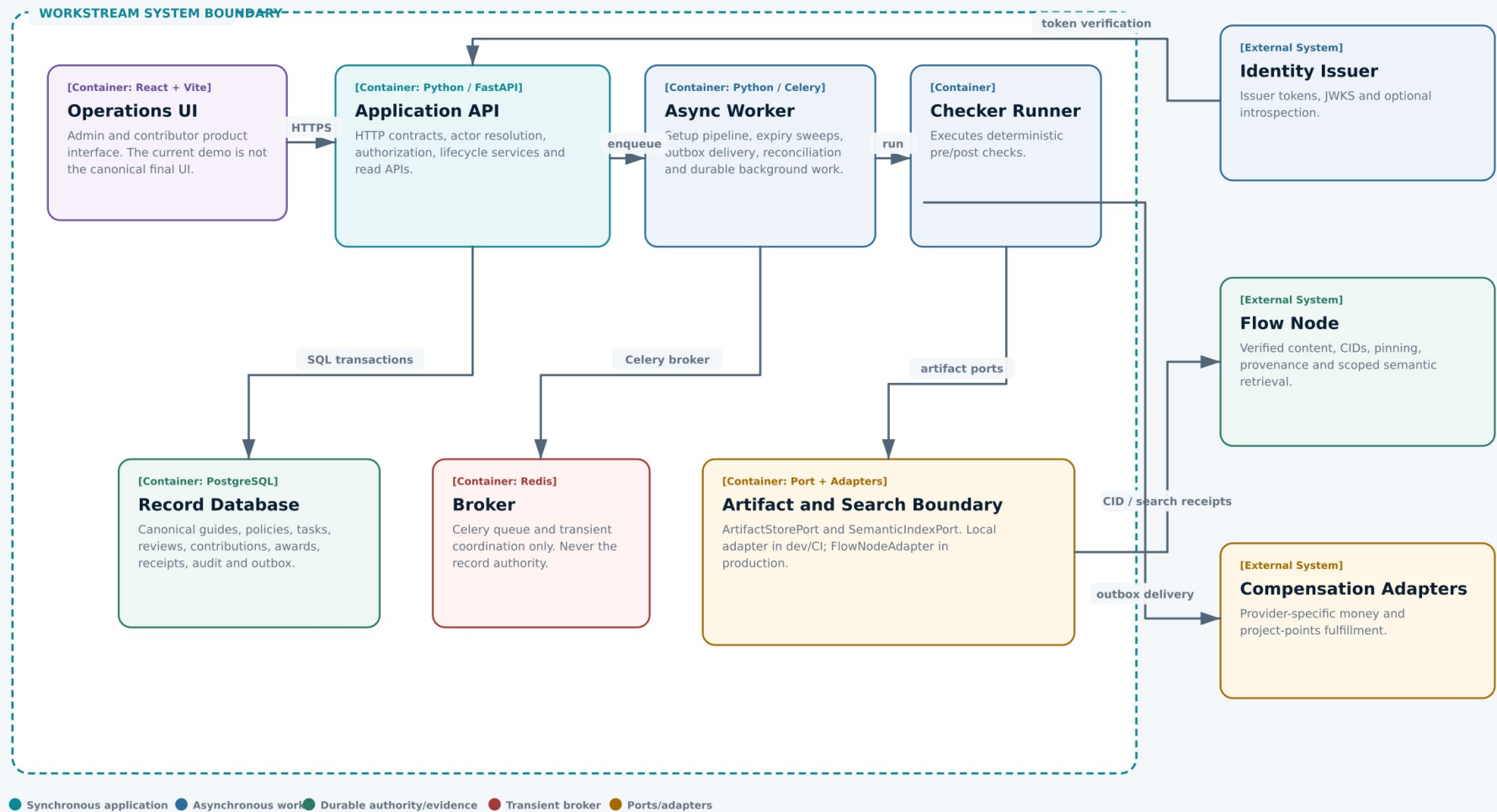
5. Workstream to compensation adapters. The transactional outbox delivers award instructions. Adapters perform provider-specific fulfillment and call back with an idempotent result. Workstream stores the receipt and updates a projection without becoming a ledger.

6. C4 Level 2 - container architecture

The v0.1 deployment is a modular monolith with separately deployable web, worker and checker processes. The separation controls latency and failure without splitting canonical state across services.

C4 Level 2 - Container View

The v0.1 modular monolith uses FastAPI, PostgreSQL, Celery/Redis and explicit external ports.



WORKSTREAM v0.1 ARCHITECTURE BASELINE

Figure 2. C4 Level 2 - Workstream containers

6.1 Container catalog

Container	Technology	Responsibility and boundary
Operations UI	React + Vite	Internal admin and contributor interface. It never writes the database directly and is not a second policy engine.
Workstream API	Python + FastAPI	HTTP contracts, token verification, local actor resolution, authorization, application services and read APIs.
Async Worker	Python + Celery	Setup jobs, checker orchestration, sweeps, outbox dispatch, reconciliation and projection rebuilds.
Checker Runner	Isolated Python process/container	Executes deterministic checker bundles against exact bound artifacts and persists structured results.
PostgreSQL	Managed PostgreSQL	Canonical lifecycle, locks, immutable chains, policy versions, contributions, awards, receipts, outbox and audit facts.
Redis	Managed Redis	Celery broker and transient coordination only; never canonical business state.
Artifact/Search Boundary	Python ports + adapters	ArtifactStorePort and SemanticIndexPort select FlowNodeAdapter in production or LocalStorageAdapter in development/CI.

6.2 Container interaction rules

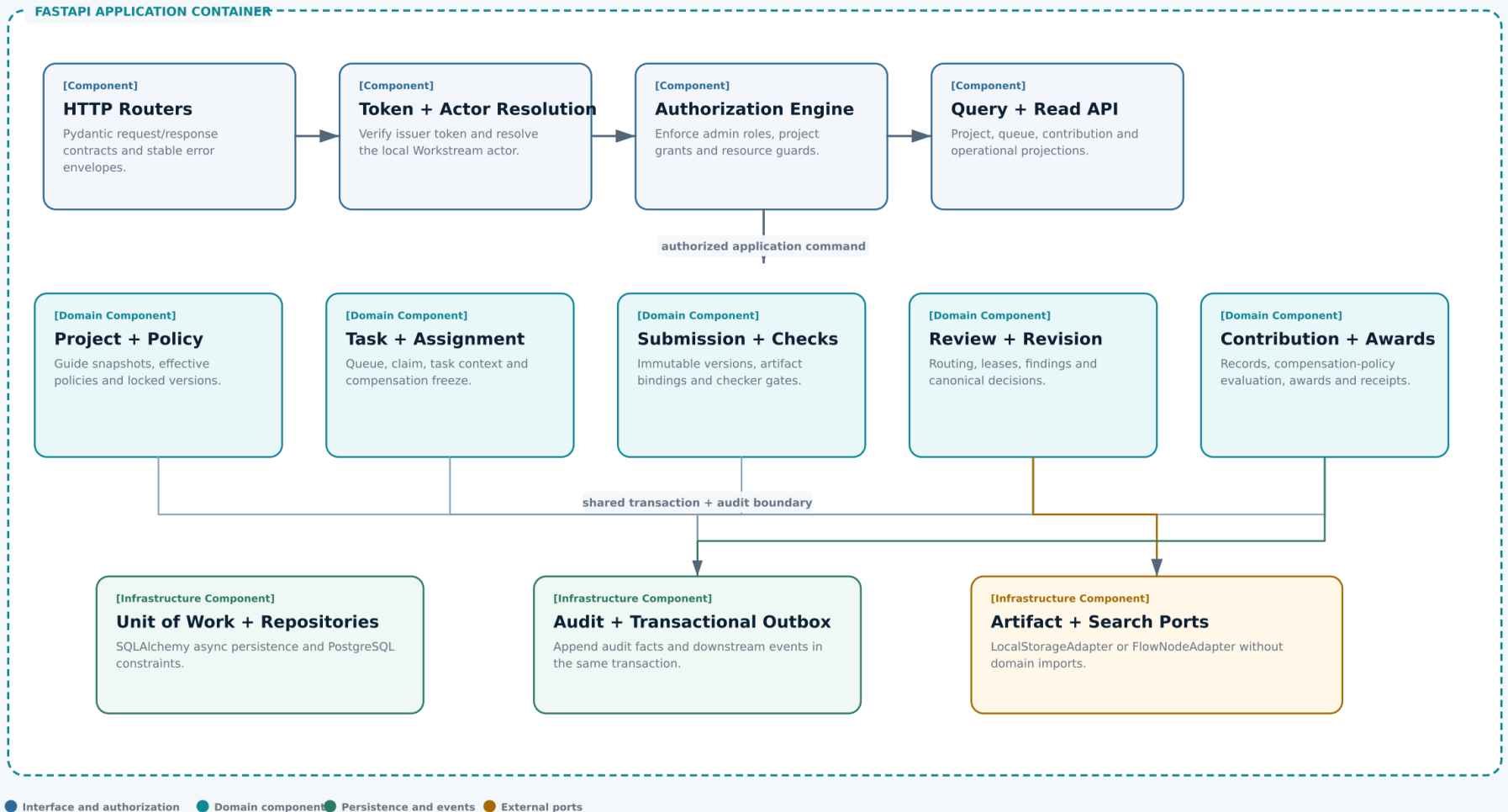
- Every business mutation enters through an application service and commits through one PostgreSQL unit of work.
- A request may enqueue a durable job or append an outbox event in the same transaction; it must not wait for provider completion.
- Redis message loss or duplication cannot create or erase canonical state; workers derive work from durable records and operate idempotently.
- Checker results are durable records tied to the checker definition and exact submission artifact bindings.
- The UI consumes supported APIs and read models; it does not encode hidden transition rules.

7. C4 Level 3 - FastAPI components

The FastAPI container is layered so transport, authority, application orchestration, domain rules and infrastructure are visible. Domain components do not import Flow Node or compensation-provider clients.

C4 Level 3 - FastAPI Application Components

Thin routers call application services; domain state changes are committed through one PostgreSQL unit of work.



WORKSTREAM v0.1 ARCHITECTURE BASELINE

Figure 3. C4 Level 3 - FastAPI application components

7.1 Request path

1. HTTP Routers validate Pydantic request contracts, route versioned endpoints and return the standard error envelope.
2. Token + Actor Resolution verifies Identity Issuer material and resolves the local Workstream actor record.
3. Authorization Engine evaluates admin roles, ProjectRoleGrant and resource-specific guards before an application command begins.
4. The application service loads the required aggregate and invokes the domain component that owns the transition.
5. The Unit of Work commits canonical rows, audit facts and outbox events atomically; the response reflects committed state only.

7.2 Domain component boundaries

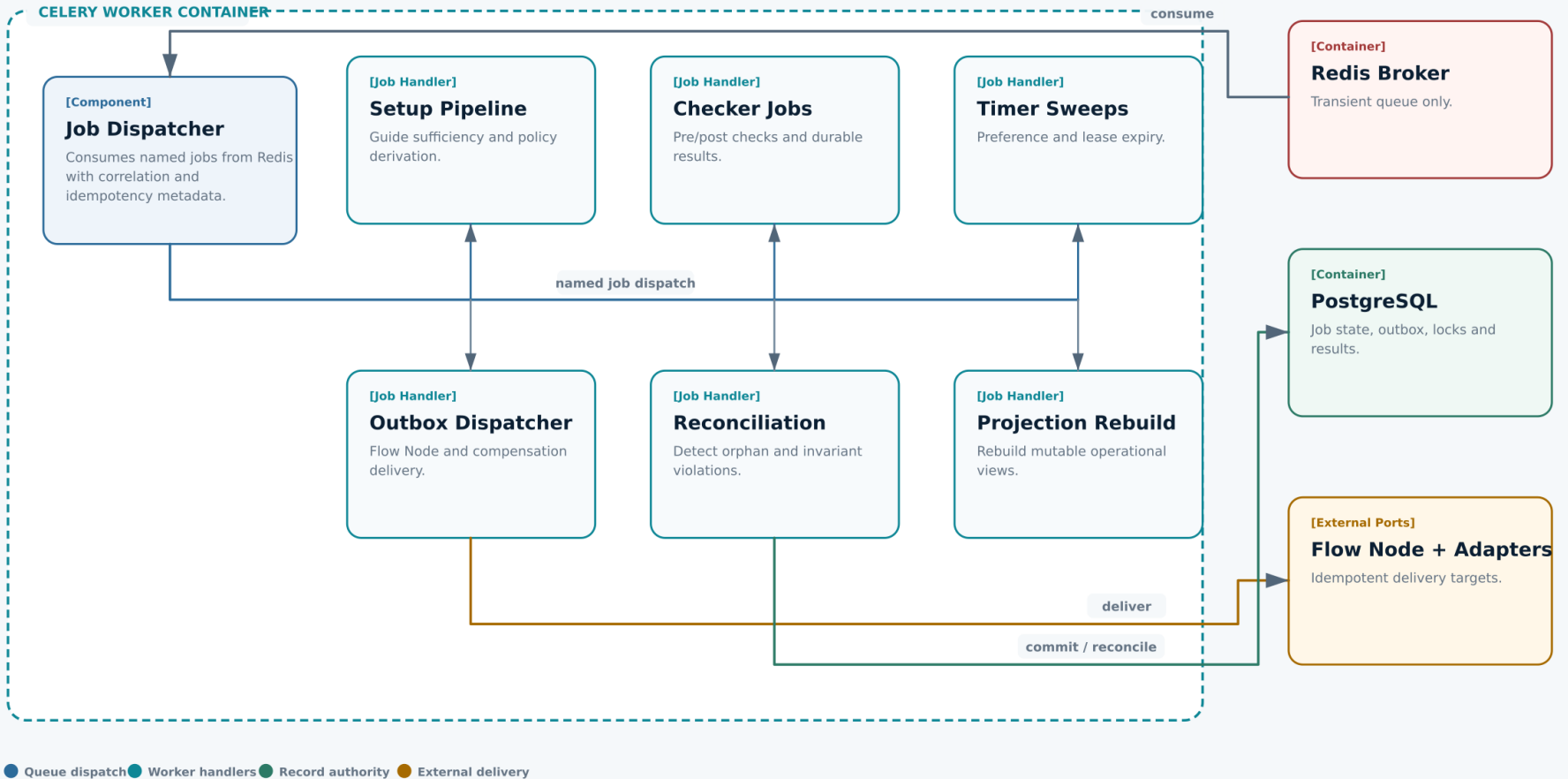
Component	Owns
Project + Policy	Guide snapshots, effective policy derivation/approval, activation and version references.
Task + Assignment	Task pool, claim, assignment, frozen policy context and submitter eligibility.
Submission + Checks	Immutable SubmissionVersion, finding responses, artifact binding and checker admission.
Review + Revision	Queue routing, preference window, leases, findings, canonical decision and revision chain.
Contribution + Awards	ContributionRecord, frozen compensation evaluation, CompensationAward, receipt projection and read composition.

8. C4 Level 3 - asynchronous worker components

Workers are durable executors, not alternate authorities. Each job reloads canonical state, rechecks preconditions and commits an idempotent outcome.

C4 Level 3 - Async Worker Components

Durable Celery jobs isolate long-running checks, timers, delivery and recovery from HTTP request latency.



WORKSTREAM v0.1 ARCHITECTURE BASELINE

Figure 4. C4 Level 3 - asynchronous worker components

8.1 Worker responsibilities

Worker component	Deterministic responsibility
Setup Pipeline	Execute guide sufficiency, policy derivation and other named project-setup stages with durable job state.
Checker Jobs	Start, monitor and persist checker runs against exact SubmissionVersion inputs.
Timer Sweeps	Expire reviewer preferences and leases using database time and locked rows.
Outbox Dispatcher	Claim undelivered events, invoke the correct port and record attempt/outcome without deleting history.
Reconciliation	Detect orphaned or contradictory operational projections and repair only replaceable state.
Projection Rebuild	Reconstruct queue, contribution or compensation status views from canonical records and receipts.

8.2 Worker invariants

- At-least-once dispatch is expected; business effects are idempotent through stable keys and database uniqueness.
- A job acknowledges only after its database outcome is committed or its retry state is durable.
- Sweep workers and request-time lazy recovery share the same transition service, preventing divergent timer behavior.
- A worker never trusts message payload state over the current locked PostgreSQL record.

9. End-to-end lifecycle

The lifecycle begins with project policy and ends at Workstream's event and receipt boundary. Green stages have live-drill evidence through review_pending. Teal stages are locked v0.1 contracts awaiting implementation proof.

End-to-End Workstream v0.1 Lifecycle

Green stages are runtime-proven through review_pending; teal stages are locked specifications awaiting implementation.

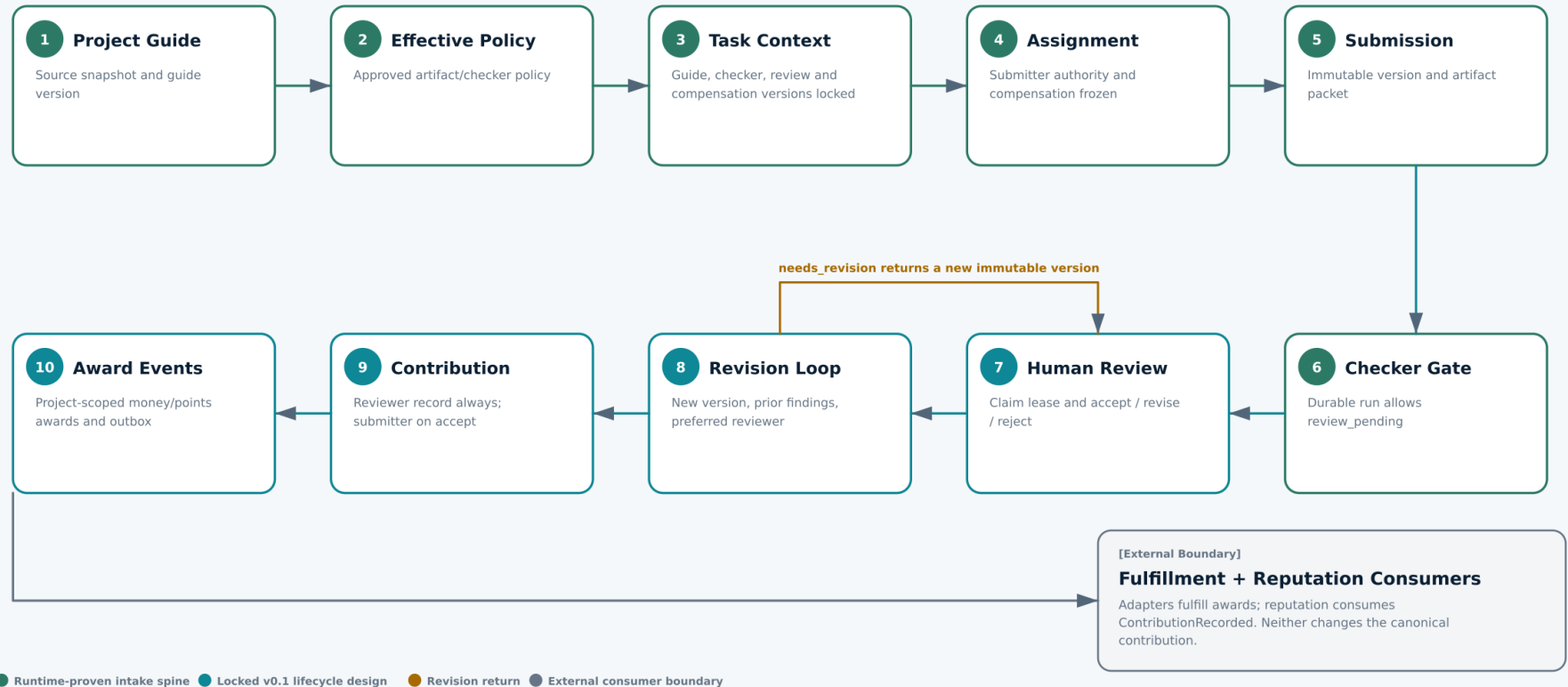


Figure 5. End-to-end Workstream v0.1 lifecycle

9.1 Lifecycle sequence

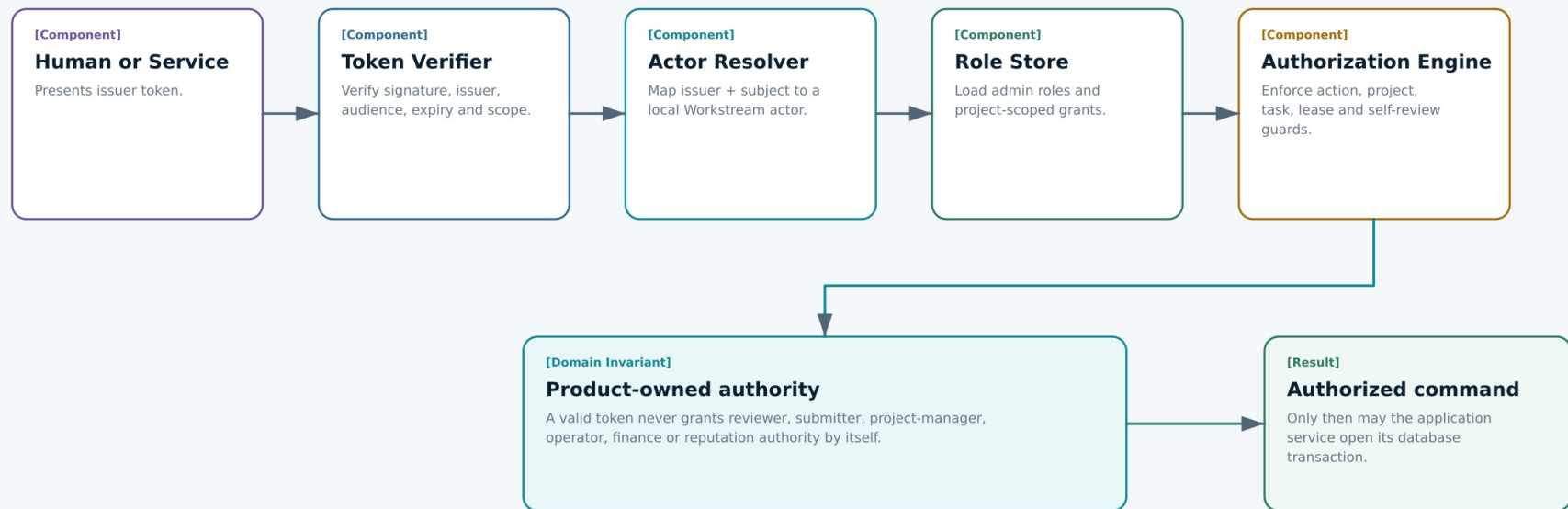
1. Project Manager creates a privacy-clean ProjectGuideSnapshot and the setup pipeline proves sufficiency.
2. Workstream derives an EffectivePolicy; the authorized admin approves and activates the exact version.
3. A task is released with locked guide, checker, review, revision and compensation policy references.
4. An eligible submitter claims the task; Workstream creates a TaskAssignment and freezes applicable terms.
5. The submitter finalizes an immutable SubmissionVersion and exact artifact intent/bindings.
6. The durable CheckerRun executes. Failure remains in the submission/checker domain; success admits the version to review_pending.
7. Workstream creates one ReviewQueueEntry. An eligible reviewer claims a time-bounded ReviewLease and records accept, needs_revision or reject.
8. needs_revision closes that queue entry but permits a new immutable version with explicit finding responses and preferred-reviewer routing.
9. Every valid recorded review creates the reviewer's ContributionRecord. Accept also creates the submitter's ContributionRecord.
10. Workstream evaluates frozen compensation policy, creates project-scoped CompensationAward records and emits outbox instructions. External consumers fulfill or score without changing canonical contribution facts.

10. Identity and authorization

Identity is intentionally separated from authority. The issuer answers who presented the request. Workstream answers whether that actor may execute this command against this resource now.

Identity and Authorization Flow

Identity Issuer proves who the actor is; Workstream proves what that actor may do to the exact resource.



● Caller ● Issuer verification ● Local actor ● Local roles ● Resource authorization

WORKSTREAM v0.1 ARCHITECTURE BASELINE

Figure 6. Identity and authorization flow

10.1 Authentication sequence

1. The browser or service presents an issuer-signed access token.
2. The Token Verifier validates signature, issuer, audience, temporal claims and accepted signing key.
3. Actor Resolver maps issuer + subject to exactly one active local Workstream actor.
4. Role Store loads active admin roles and project-scoped ProjectRoleGrant records.
5. Authorization Engine applies command-specific role and resource guards, including self-review and banned-assignment checks.
6. Only an authorized command opens the application transaction.

10.2 Identity invariants

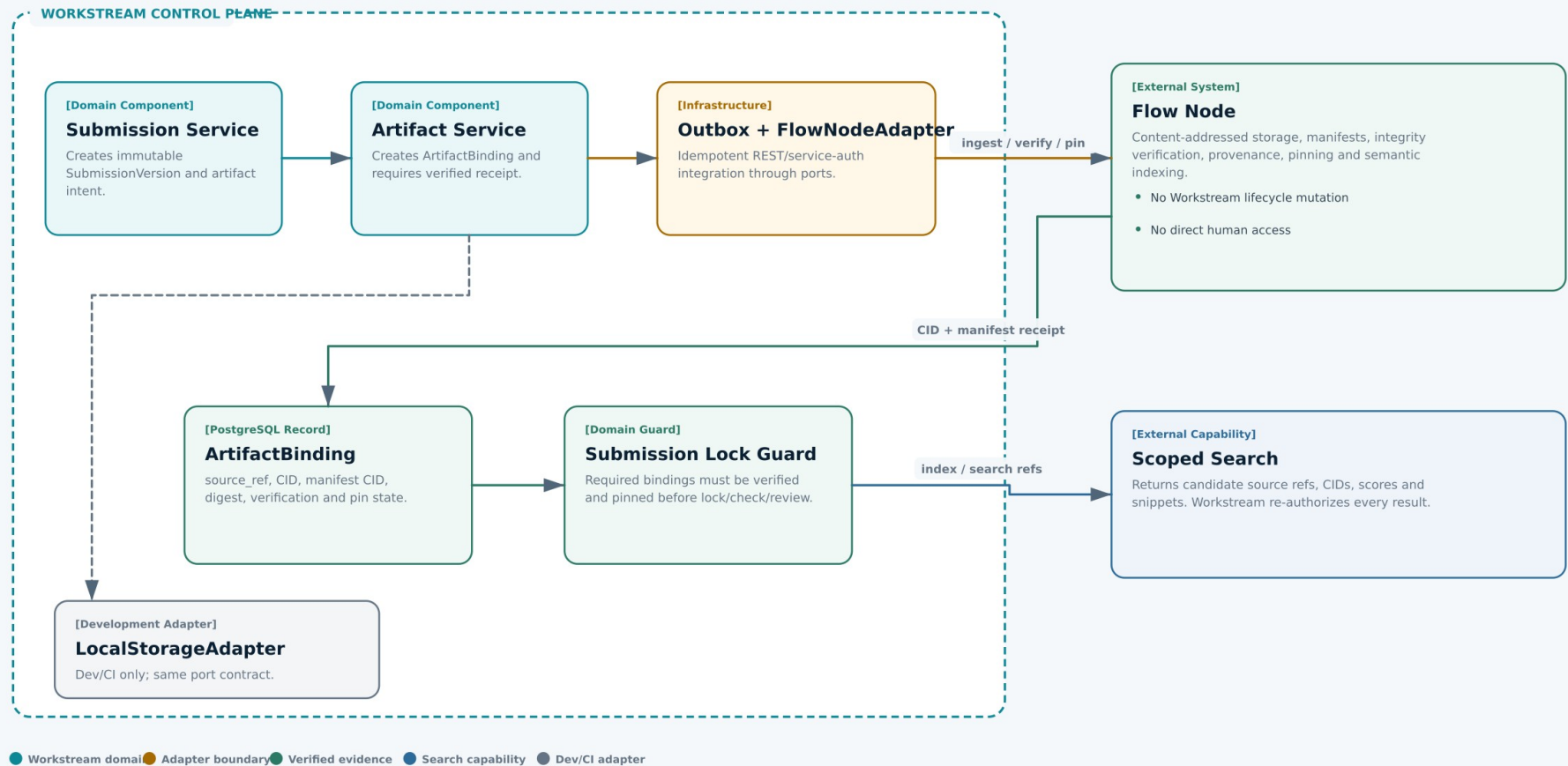
- Tokens must not contain authoritative Workstream reviewer, submitter, finance or project-manager grants.
- Role revocation takes effect from Workstream state even while a previously issued identity token is still cryptographically valid.
- Unknown issuer subjects are denied until a deliberate local actor-linking/onboarding action occurs.
- Service-to-service calls use a separate service principal and audience; human bearer tokens are not forwarded to Flow Node or compensation adapters.

11. Flow Node evidence boundary

Workstream and Flow Node divide meaning from evidence. Workstream owns why an artifact matters and what lifecycle decisions reference it. Flow Node owns immutable content, manifest, integrity verification, provenance, pinning and search indexes.

Workstream - Flow Node Evidence Boundary

Flow Node strengthens evidence identity and retrieval; PostgreSQL remains the authority for lifecycle meaning.



WORKSTREAM v0.1 ARCHITECTURE BASELINE

Figure 7. Workstream - Flow Node evidence boundary

11.1 Evidence flow

1. Submission Service creates the immutable SubmissionVersion and records artifact intent in the Workstream transaction.
2. Artifact Service creates or updates the operational ArtifactBinding state and appends an idempotent outbox command.
3. FlowNodeAdapter sends service-authenticated ingest/verify/pin work to Flow Node.
4. Flow Node returns a content identifier, manifest identifier, digest verification and pin state.
5. Workstream stores the receipt in ArtifactBinding; a required binding must be verified and pinned before lock, checker admission or human review.
6. Semantic search returns candidate references and scores. Workstream re-authorizes every result for the requesting actor and project before disclosure.

11.2 Port contract

Port	Production adapter	Development/CI adapter	Rule
ArtifactStorePort	FlowNodeAdapter	LocalStorageAdapter	Same domain contract; no Flow Node imports inside domain services.
SemanticIndexPort	FlowNodeAdapter	In-memory/local test adapter	Search results are candidates and always pass Workstream authorization.

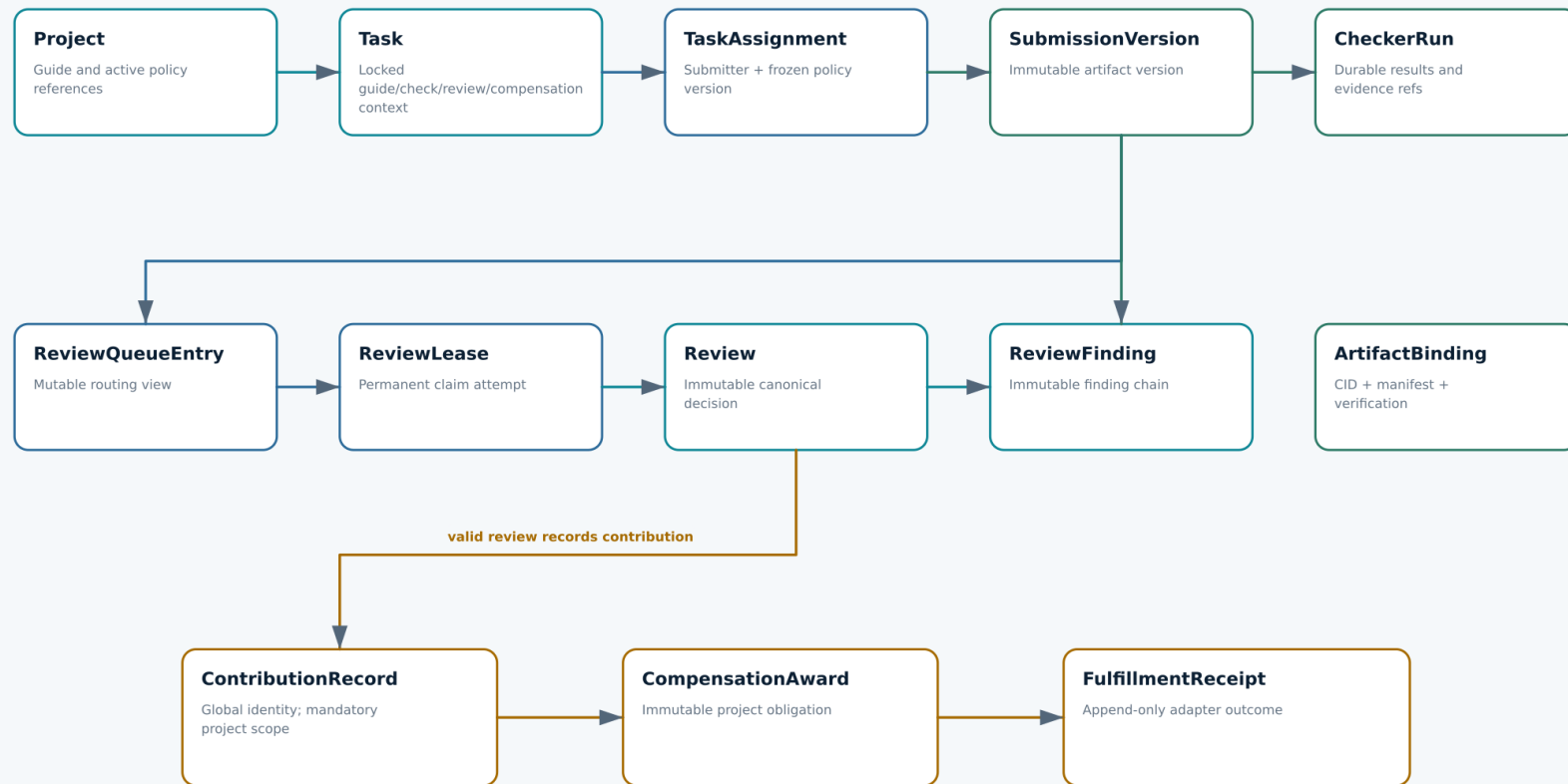
FAILURE RULE Flow Node unavailability is infrastructure/evidence-unavailable state. It may pause lock, checking or review, but it must never be translated into contributor rejection.

12. Canonical domain and data ownership

The canonical model links immutable evaluation objects to operational routing objects. Routing and projections may change; submitted evidence, reviews, contributions, awards and receipts are append-only facts.

Core Domain Relationship Map

Canonical lifecycle records are linked by immutable identifiers; operational routing and projections remain replaceable.



● Canonical domain ● Operational routing/assignment ● Evidence/checks ● Contribution/compensation

WORKSTREAM v0.1 ARCHITECTURE BASELINE

Figure 8. Core domain relationship map

12.1 Canonical versus operational records

Class	Records	Mutation rule
Canonical policy	ProjectGuideSnapshot, EffectivePolicyVersion, CompensationPolicyVersion	Published/activated versions are immutable; new policy creates a new version.
Canonical evaluation	SubmissionVersion, CheckerRun result, Review, ReviewFinding, FindingResolution	Append-only or immutable after finalization; history is never overwritten.
Canonical outcome	ContributionRecord, CompensationAward, CompensationFulfillmentReceipt, audit event	Immutable business facts. Corrections are new records or explicit superseding facts.
Operational routing	Task queue view, ReviewQueueEntry, ReviewLease, job state	Mutable under locked transitions; reconstructable from canonical history where specified.
Projection	Queue counts, compensation status, contribution views	Replaceable and rebuildable; never used as the sole authority for a transition.

12.2 Ownership matrix

Data	Canonical owner	External reference retained by Workstream
Local actor, admin role, ProjectRoleGrant	Workstream	issuer + subject linking reference
Task, assignment, submission, checker, review	Workstream	artifact CIDs/manifests where applicable
Artifact bytes, manifests, provenance, pin	Flow Node	ArtifactBinding with CID, manifest CID, digest and verification/pin state
Contribution and compensation award	Workstream	adapter binding and delivery identifiers
Payment/points execution and ledger	External adapter/provider	immutable fulfillment receipt and provider reference

13. Review, revision and contribution lifecycles

13.1 Review routing and lease model

WS-REV-001 is normative. Work enters review only after checker admission. The queue is disposable routing; the submission and review chains are permanent.

- One ReviewQueueEntry routes exactly one SubmissionVersion.
- An eligible reviewer requires an active ProjectRoleGrant with reviewer or both for that project.

- First submissions enter open FIFO. Revision submissions prefer the prior reviewer for a configured window.
- A reviewer may hold no more than the configured active-lease capacity; v0.1 defaults to one active review at a time.
- Lease expiry or manual release returns the entry to open FIFO, clears stickiness and records a distinct audit event.
- Decision and expiry race under database locking; exactly one terminal outcome wins.

13.2 Decision semantics

Decision	Required effect
accept	Create immutable Review, close queue/lease, close successful task chain, create reviewer contribution and submitter contribution, then evaluate frozen awards.
needs_revision	Require non-empty findings, create immutable Review, close current queue/lease, create reviewer contribution, and permit a later new SubmissionVersion with finding responses.
reject	Create immutable Review, close queue/lease, ban that contributor's TaskAssignment for the task, close the task in v0.1, and create reviewer contribution only.

13.3 Contribution and compensation boundary

WS-CON-001 is normative. Every valid reviewer decision creates a reviewer ContributionRecord regardless of accept, needs_revision or reject. The submitter receives a ContributionRecord only when the submission is accepted.

- ContributionRecord identity is global and can be filtered by project; economic terms and awards are project-scoped.
- The applicable CompensationPolicyVersion is frozen at assignment or review-lease creation as specified, preventing retroactive rate changes.
- A single contribution may produce zero, one or several award instruments, such as money and points, according to the frozen policy.
- Workstream ends at CompensationAwardCreated, CompensationFulfillmentRequested and the immutable callback receipt/status projection.
- External adapters own payment requests, provider attempts, finance workflow, retries, wallets, balances and point ledgers.

14. Transactions, concurrency and events

14.1 Transaction boundary

ATOMICITY RULE A lifecycle transition, its audit fact and every downstream outbox event caused by that transition commit in one PostgreSQL transaction. Network delivery occurs later.

Application services use a Unit of Work. The service locks the minimal aggregate rows, rechecks authorization-sensitive state, validates the domain transition, writes canonical facts, updates operational state and appends outbox records before commit.

14.2 Required race handling

Race	Required database behavior
Two reviewers claim one entry	Lock the pending entry; one lease insert/update succeeds and the other receives a deterministic conflict.
Reviewer capacity	Lock or constraint the reviewer's active-capacity record so concurrent claims cannot exceed policy.
Decision versus lease expiry	Lock lease and queue rows; one transition wins, the other observes terminal state and becomes idempotent/conflict as specified.
Duplicate decision request	Idempotency key plus unique Review per SubmissionVersion returns the original committed result when payload matches.
Policy activation versus assignment	Lock project policy selector so TaskAssignment freezes exactly one active version.
Duplicate adapter callback	Unique adapter/callback identity returns the existing receipt; contradictory terminal result is rejected and audited.

14.3 Event architecture

Domain events are persisted in the transactional outbox. The dispatcher publishes or invokes an adapter at least once. Consumers use event_id or a stable business idempotency key to produce an effectively-once business effect.

- ContributionRecorded is the canonical signal for future reputation processing.
- CompensationAwardCreated describes the immutable award; CompensationFulfillmentRequested is the adapter instruction.
- ReviewerLeaseExpired and ReviewerReleasedTask are separate audit facts so future reputation policy can distinguish timeout from voluntary release.
- No consumer response may edit the source Review or ContributionRecord.

15. API and integration architecture

15.1 HTTP conventions

- All product APIs are versioned under /v1. This is an API compatibility namespace and does not change the product version from Workstream v0.1.
- Pydantic request/response contracts reject unknown invalid states and return one stable error envelope with code, message, details, correlation_id and optional retry metadata.
- Mutation endpoints accept an idempotency key where client replay is expected; keys are scoped to actor, operation and resource.

- ETag/version checks or row-version predicates protect admin updates that do not use immutable version records.
- Read endpoints compose canonical records with replaceable projections but label external fulfillment state clearly.

15.2 Boundary contracts

Boundary	Protocol	Trust and idempotency
Browser/UI to API	HTTPS JSON	Issuer access token, CSRF/origin controls where applicable, request idempotency for mutations.
API/worker to PostgreSQL	TLS SQL	Least-privileged service role; transaction and constraint authority.
API/worker to Redis	TLS broker	Transient job envelope only; no human token or canonical state.
Workstream to Identity Issuer	HTTPS JWKS/introspection	Pinned issuer/audience; cached JWKS with safe overlap and failure policy.
Workstream to Flow Node	HTTPS REST/service auth	Service principal, stable ingest key, digest verification and receipt reconciliation.
Workstream to compensation adapter	HTTPS/event delivery + callback	Per-binding credentials, award idempotency key, signed callback and replay protection.

16. Security, privacy and audit

16.1 Security controls

- Deny by default for unknown actors, missing grants, inactive projects, self-review, banned assignments and stale/terminal resources.
- Separate human, worker and external-adapter principals; never forward a human bearer token across service boundaries.
- Encrypt connections and managed storage; store secrets in the runtime secret manager, not project policy or event payloads.
- Checker runner receives a minimal read-only input package, resource/time limits and no ambient production credentials.
- Artifact and search responses are re-authorized by Workstream and filtered to project/resource scope.
- Admin overrides require a reason, actor, before/after state and audit event; high-risk overrides may require a second authority when policy enables it.

16.2 Privacy and minimization

Workstream stores only identity-linking references and product profile data required for authorization and contribution display. Event payloads and artifact manifests exclude credentials, provider secrets and unnecessary personal data. Global contribution views are

authorization-filtered and project details are disclosed only when the viewer may see that project.

16.3 Audit requirements

Every sensitive transition records who acted, under which local role/grant, against which project/resource, at what time, with which correlation and idempotency identifiers, and which immutable versions were in force. Audit history is append-only and queryable without relying on application logs.

17. Failure handling, recovery and observability

17.1 Failure classification

Failure class	Lifecycle response
Client/authority failure	Reject before mutation with stable 4xx error; audit sensitive denial where policy requires.
Domain conflict	Return deterministic 409/422; do not partially transition.
Checker failure	Persist structured CheckerRun outcome or infrastructure error; never invent a human review decision.
Evidence unavailable	Pause dependent gate and retry/reconcile; never reject the contributor for provider unavailability.
Worker/broker failure	Recover from durable job/outbox state; redelivery is safe.
Adapter failure	Keep award canonical and status pending/failed as receipt semantics require; retry outside the originating request.
Projection corruption	Rebuild projection from canonical rows and receipts.

17.2 Recovery mechanisms

- Lazy timer recovery on read/claim plus scheduled preference and lease sweeps.
- Outbox retry with exponential backoff, attempt history and dead-letter operational view.
- Artifact reconciliation compares local ArtifactBinding state with Flow Node receipt/proof state.
- Contribution/award reconciliation detects accepted reviews lacking required records and repairs only by the normative atomic service or explicit audited repair command.
- Database backup/restore drills include immutable chains, outbox, audit and projection rebuild verification.

17.3 Observability

All processes emit structured logs, metrics and traces keyed by `correlation_id`, `project_id`, `task_id`, `submission_version_id`, `review_id`, `contribution_id` and `award_id` where applicable. Logs must not contain artifact bytes, access tokens or compensation-provider secrets.

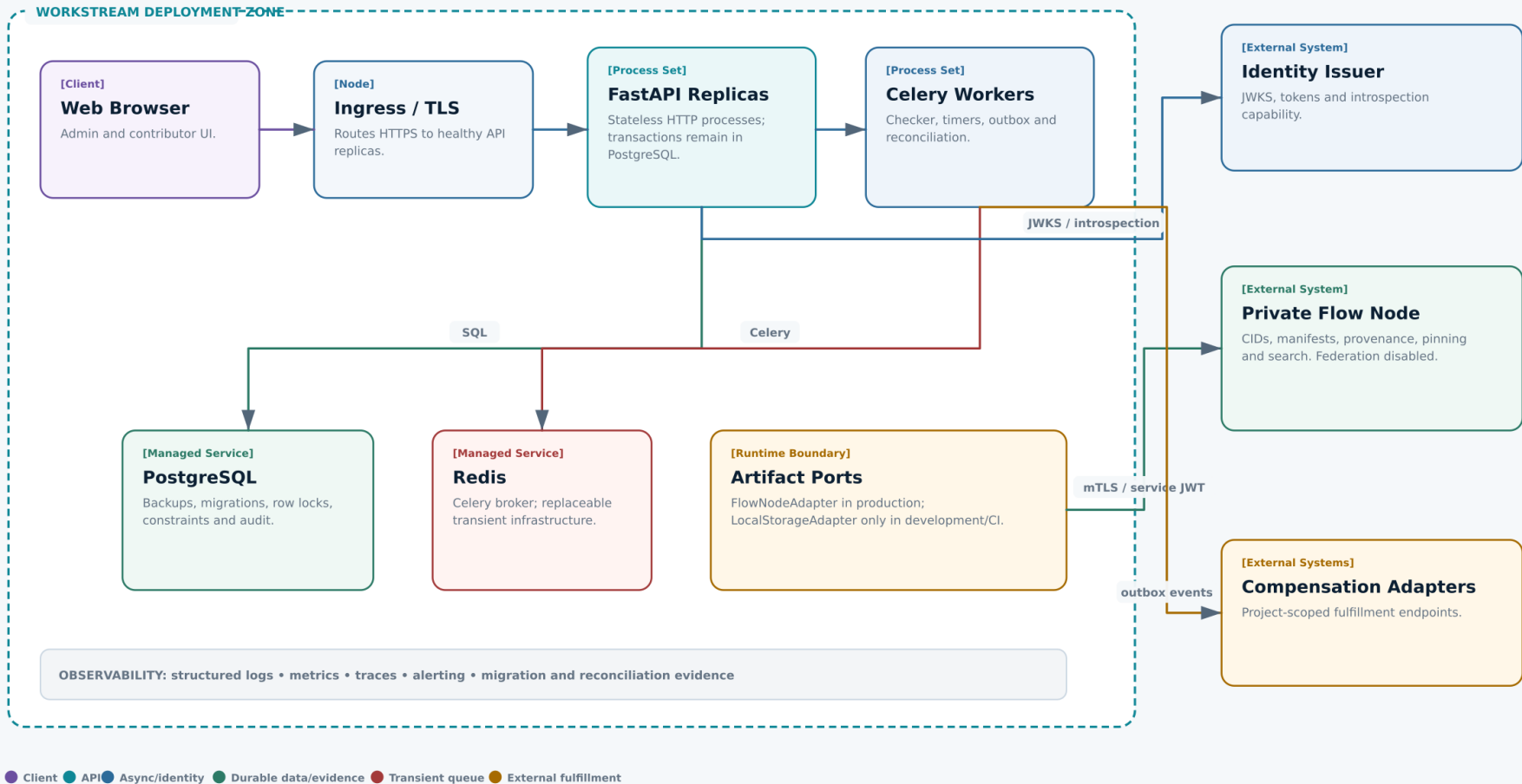
Signal	Minimum measures
API	Latency/error by route and <code>error_code</code> ; authorization denials; idempotent replay rate.
Queues	Pending/open/preferred age, active leases, lease expiry/release rate, claim conflicts.
Checker	Run duration, result class, infrastructure error, backlog and retry count.
Outbox	Oldest undelivered age, attempts, success/failure by adapter and dead-letter count.
Evidence	Ingest/verify/pin latency, unavailable binding count and reconciliation drift.
Contribution	Reviews without expected contribution, awards awaiting fulfillment and contradictory callback attempts.

18. C4 deployment view

The private production default uses replicated stateless API and worker processes around managed PostgreSQL and Redis. External systems remain behind authenticated, observable boundaries.

C4 Deployment View - Private Production Default

Replicated API and worker processes share PostgreSQL authority; external capabilities remain behind authenticated service boundaries.



WORKSTREAM v0.1 ARCHITECTURE BASELINE

Figure 9. C4 deployment view - private production default

18.1 Deployment rules

- FastAPI replicas are stateless; local filesystem is not canonical production storage.
- Worker pools may scale by queue, but all use the same versioned application/domain packages as the API deployment.
- Checker runners are isolated from API/worker credentials and use bounded CPU, memory, time and network policy.
- Database migrations are forward-only, reviewed and applied as a controlled release step before code requiring the new schema.
- Deployment health distinguishes process health from dependency readiness; a Redis outage must not make PostgreSQL state unreadable.
- Production uses FlowNodeAdapter. LocalStorageAdapter is an explicit development/CI configuration and must fail startup if selected in production.

19. Conformance, delivery and baseline decisions

19.1 Required conformance suites

Suite	Exit evidence
Identity	JWKS rotation, invalid issuer/audience, unknown subject, role revocation and service-principal tests.
Intake	Existing live drill remains green through review_pending with no failed-preflight side effects.
Review	All WS-REV-001 authority, FIFO, preference, lease, decision, revision, reject and race tests pass.
Contribution	All WS-CON-001 creation matrix, frozen policy, award, outbox, callback and contradiction tests pass.
Evidence	Flow Node ingest, digest mismatch, pin failure, retrieval, search authorization and recovery tests pass.
Recovery	Broker redelivery, worker crash, outbox replay, projection rebuild and backup/restore drills pass.

19.2 Delivery sequence

Phase	Deliverable	Exit condition
0	Approve WS-ARCH-001 and retain WS-REV-001/WS-CON-001 as normative child specifications.	One Workstream-only decision set.
1	Identity verifier, actor resolution and role/grant enforcement.	Identity conformance green.

Phase	Deliverable	Exit condition
2	Review schema, queue, leases, decisions, revision chain and recovery.	WS-REV-001 conformance and drill green.
3	Contribution, compensation policy, awards, outbox and callbacks.	WS-CON-001 conformance and drill green.
4	Production FlowNodeAdapter and evidence gates.	Evidence conformance and recovery green.
5	Operations/read views, observability, backup/restore and production hardening.	Full accepted-work drill and runbook sign-off.

19.3 Locked architecture decisions

ADR	Decision
ADR-01	Workstream v0.1 is a Python/FastAPI modular monolith; no Rust/PyO3 lifecycle split.
ADR-02	PostgreSQL is canonical lifecycle authority; Redis and projections are replaceable.
ADR-03	Identity Issuer proves identity; Workstream owns all product roles and resource authorization.
ADR-04	Flow Node owns immutable evidence capabilities; Workstream owns lifecycle meaning and binding rules.
ADR-05	Production uses FlowNodeAdapter; LocalStorageAdapter is development/CI only.
ADR-06	Long-running work uses Celery and transactional outbox; request paths do not wait on external fulfillment.
ADR-07	Review and submission chains are immutable; queue and projection state are operational.
ADR-08	Reviewer contributions are outcome-neutral; submitter contribution requires accept.
ADR-09	Compensation awards are Workstream facts; money/points execution and ledgers are external.
ADR-10	Reject is task-terminal in v0.1 and bans that contributor-task assignment.
ADR-11	Evidence-provider failure pauses work and cannot become contributor rejection.
ADR-12	Every transition, audit fact and caused outbox event commits atomically.

19.4 Definition of baseline completion

- Every architecture boundary is represented in code by an interface, module boundary, authorization rule or deployment policy.
- Every canonical transition is transactional, auditable and tested under concurrency and replay.
- Every required artifact is verified and pinned before it becomes evaluation evidence.
- Every review and contribution rule matches WS-REV-001 and WS-CON-001 without local reinterpretation.

- The full internal task path completes from project guide through adapter receipt with observable evidence and no hidden manual database edits.

Appendix A. Glossary

Term	Definition
Canonical	Authoritative business fact used to decide lifecycle truth.
Projection	Replaceable read or status view derived from canonical facts.
ProjectRoleGrant	Admin-issued, project-scoped contributor privilege: submitter, reviewer or both.
SubmissionVersion	Immutable finalized submission in a monotonic task chain.
ReviewQueueEntry	Mutable routing record for one SubmissionVersion.
ReviewLease	Time-bounded exclusive claim to record a review decision.
ContributionRecord	Immutable global identity of a recognized submitter or reviewer contribution, filterable by project.
CompensationAward	Immutable project-scoped result of evaluating frozen compensation policy for a contribution.
Fulfillment receipt	Immutable Workstream record of an authenticated external adapter response.
ArtifactBinding	Workstream record connecting a lifecycle object to Flow Node evidence identifiers and verification/pin state.
Transactional outbox	Database records committed with business state and delivered asynchronously at least once.

FINAL BOUNDARY Workstream governs the work. Identity Issuer proves the actor. Flow Node preserves the evidence. External adapters fulfill the award. PostgreSQL records what happened and why.